

从零构建 Web CAD : B-Rep、原生文档与 STEP 互操作

Architecture exploration note

核心判断

Web 端可以把精确 B-Rep 建模与开放格式互操作拆成两个层次：原生文档负责持续编辑与恢复状态，STEP 负责跨 CAD 系统的导入/导出。两者不应该被强行合并成同一种文件格式。

本文记录当前讨论形成的技术方向：浏览器内运行几何内核（例如 OCCT/WASM），以 B-Rep 作为几何真相源，以独立的 document model 保存建模、配元据，在用需要通 translation layer 导入或导出 STEP。

一页结论

问题	建议
Web 中的 canonical geometry	让 B-Rep 保持在 WASM/kernel ；用只持有定 handle / ID。
原生 document	不要直接等同于 STEP；保存 feature/assembly/metadata + B-Rep checkpoint + 可选 preview。
B-Rep 持久化	OCCT 技术栈早期可直接使用 OCCT BRep/native serialization；长期可放进 versioned binary payload。
STEP	作为 interchange boundary：导入时翻译成 canonical B-Rep，导出时从当前 B-Rep 写 STEP。
复杂 STEP	从一开始考虑 XDE/XCAF，而不只是裸 TopoDS_Shape，以保留 assembly、name、color、units 等语义。
性能	避免 B-Rep 在 JS heap 和 WASM memory 之间反复复制；tessellation 只用于显示。

1. 目标与非目标

目标是构建一个真正的 Web solid modeler：用户在浏览器中从零创建和修改精确几何，同时可以在需要时导入和导出 STEP。渲染不是主问题；核心是几何内核、文档语义、持久化和 translation 的边界。

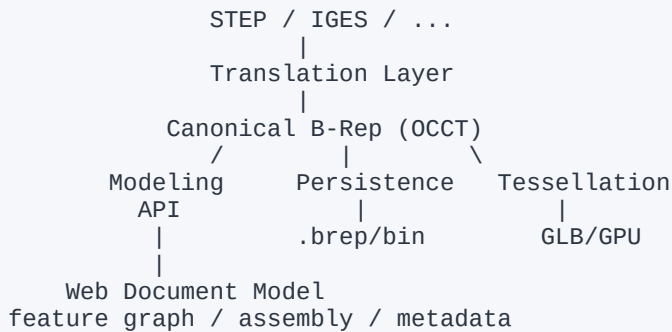
建议的最小能力闭环：

- 创建 primitive / sketch-derived solid
- Boolean / Extrude / Fillet 等 B-Rep 操作
- 快速 tessellation 到 WebGL/WebGPU viewport
- 保存原生 Web CAD document
- 导入 STEP，并在浏览器中继续编辑
- 导出当前模型为 STEP / GLB / STL

非目标

第一阶段不需要 reverse-engineer 一个 STEP 文件原本的 feature history，也不需要一开始实现 Fusion/Onshape 级参数化能力。导入的 STEP 可以作为 ImportedBody，之后继续叠加 Web 端新建的 feature。

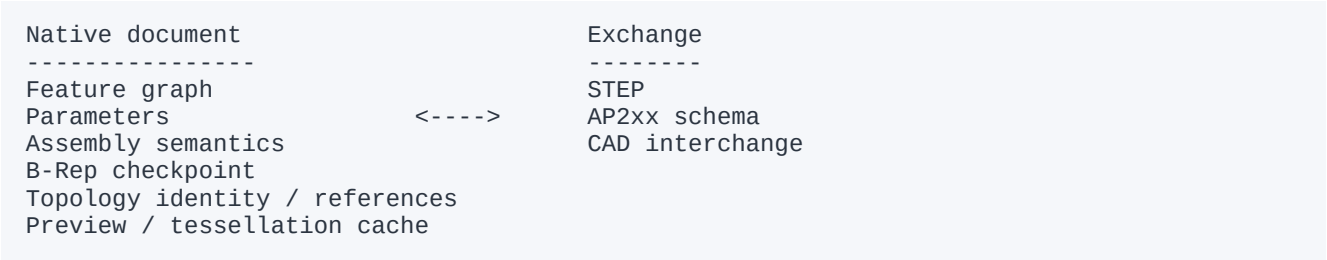
2. 总体架构



这里最重要的原则是：translation 是边界能力，而不是应用内到处存在的概念。无论几何来自 Web 原生建模还是 STEP import，进入系统后都应该成为同一种 canonical B-Rep。

3. 为什么原生 document 不应该直接使用 STEP

STEP 的优势是中立、标准化和跨 CAD kernel 互操作；但它并不天然等价于你的 Web CAD authoring document。原生文档还需要保存参数、feature graph、装配实例、选择语义、版本、缓存、UI-independent metadata 等。



因此一个较稳妥的原则是：**document format 是 container；B-Rep serialization 是其中一个 payload；STEP 是外部 interchange representation。**

4. B-Rep 应该如何存储

如果几何 kernel 是 OCCT，那么早期 prototype 使用 OCCT 自己的 BRep/native serialization 是最简单的选择。它可以直接恢复 TopoDS_Shape 以及底层曲线、曲面和拓扑关系，不需要每次保存和打开都做 STEP round-trip。

格式	精确 B-Rep	内部存储	互操作	推荐角色
OCCT .brep	是	高	低	OCCT native snapshot
STEP	是	中	高	外部交换
Parasolid x_t/x_b	是	高	高	Parasolid 栈
GLB	否 (mesh)	低	高	预览/实时渲染

推荐的容器化结构

```
model.webcad
  manifest.json      # schemaVersion, units, kernelVersion
  features.json/bin  # feature graph / parameters
  geometry.brep/bin  # exact B-Rep checkpoint
  topology.bin       # optional persistent identity mapping
  preview.glb        # optional fast preview
```

这种设计让 document format 与具体 kernel serialization 解耦。未来升级 OCCT 版本、替换 B-Rep 编码、增加装配语义时，不需要推翻整个文件格式。

5. STEP import / export 应该如何进入系统

STEP import 应被视为一个明确的 translation transaction。导入完成以后，上层应用只看到 canonical bodies / parts，而不是 STEP entity。

```
.step bytes
-> STEP reader
-> transfer / unit handling
-> healing / validation
-> TopoDS_Shape / XCAF document
-> register geometry handles
-> tessellate
-> GPU viewport
```

STEP export 的方向相反：从当前 canonical B-Rep 生成 STEP representation，并把结果作为 Blob / File System API 输出。用户在 Web 中做过 Boolean、Fillet、Transform 等修改后，导出的应是修改后的最终精确几何。

```
STEP import -> canonical B-Rep -> Web edits -> canonical B-Rep -> STEP export
```

6. 导入 STEP 后如何继续编辑

不要默认把 STEP 还原成它在原 CAD 软件里的 sketch/extrude/fillet history。多数情况下 STEP 只给你结果几何和产品，不提供可的原始 parametric feature history。

```
Document
  ImportedBody_01    # source = STEP
    exact B-Rep
  Feature_01         # Boolean Cut added in Web
  Feature_02         # Fillet added in Web
  CurrentBody        # current canonical B-Rep
```

这使系统更接近 direct modeling：导入体是一个稳定的 base feature，之后的 Web 操作建立在它上面。如果未来需要 feature recognition，可以作为独立能力加入，而不是 STEP import 的前置条件。

7. 真正困难的问题：拓扑引用与重算

复杂 B-Rep 的困难通常不是“文件够不够小”，而是 feature 之间如何稳定引用 face/edge。一个 Boolean 或尺寸化可能改拓，因此直接保存 Face_17 这种引用并不可靠。

```
Bad:   Fillet -> Face_17
Better: Fillet -> semantic / persistent reference
       e.g. "top planar face produced by Extrude_03"
```

因此中长期需要 persistent naming / topology identity mapping。对于第一版，可以先限制 feature 重算范围，或把导入体视为 immutable base，再在其上做 direct edits，从而推迟最困难的参数化依赖问题。

8. WebAssembly 边界

B-Rep 应尽量长期驻留在 WASM linear memory / kernel heap 中。JavaScript/TypeScript 层通过 handle 调用建模操作，避免把复杂拓扑与曲面数据反复 materialize 成 JS object。

```

JavaScript / TypeScript
  BodyId / FaceId / OperationId
    | API boundary
    v
WASM / OCCT
  TopoDS_Shape
  BRepAlgoAPI_*
  BRepMesh_*
  STEP / XCAF translators

```

例如 application API 更适合是 kernel.booleanCut({target, tool})，而不是把一整套 B-Rep 拷到 JS 后再处理。tessellation 输出 mesh buffer，才是适合送到 WebGL/WebGPU 的数据。

9. TopoDS_Shape 还是 XCAF/XDE

单零件 prototype 只使用 TopoDS_Shape 足够；但如果希望导入真实世界的 STEP assembly，建议尽早把 XDE/XCAF 纳入 document 边界。复杂 STEP 不只有几何，还可能包含 assembly hierarchy、part names、colors、layers、units 和产品结构。

```

Web CAD Document
  Product / Scene Tree
    Assembly
      Instance -> Part
  Metadata
    name / color / units / layer
  Geometry
    handle -> TopoDS_Shape

```

建议

如果未来明确要支持 assembly STEP，请把“CAD document model”与“shape model”从第一版就分开。否则从单纯 TopoDS_Shape 升级到产品结构模型时，改动会集中爆发。

10. 推荐 MVP

一个能验证架构的第一版不需要很大。关键是把完整生命周期走通，而不是堆很多建模命令。

阶段	能力	验证问题
MVP-0	Box/Cylinder + Boolean + viewport	OCCT/WASM 与渲染边界是否可行
MVP-1	保存 native document + .brep checkpoint	浏览器重启后的恢复成本
MVP-2	STEP import + edit + STEP export	translation round-trip 与几何质量
MVP-3	assembly/name/color via XCAF	真实 STEP 的文档语义保留
MVP-4	persistent references / incremental recompute	参数化编辑的稳定性

11. 当前建议的技术决策

- Geometry kernel：OCCT compiled to WebAssembly（或等价的精确 B-Rep kernel）。
- Canonical geometry：kernel-native B-Rep，不使用 mesh 作为几何真相源。
- Native document：versioned container，保存 feature/assembly/metadata + B-Rep checkpoint。
- B-Rep persistence：prototype 阶段直接使用 OCCT .brep/native serialization；后续可换成 versioned binary payload。
- Interchange：STEP 是必须支持的第一优先级；GLB/GLTF 用于显示/实时生态，STL 用于简单 mesh interchange。
- Imported STEP：作为 ImportedBody/base feature，而不是自动伪造 parametric history。
- Assembly：如果是目标范围，优先考虑 XCAF/XDE，而非只用 TopoDS_Shape。
- Web boundary：B-Rep 常驻 WASM；JS 只持有 handles、document semantics 与渲染 buffer。

12. 开放问题

- 复杂模型在浏览器中 serialize / deserialize 的实际吞吐量和峰值内存是多少？
- IndexedDB / OPFS —更适合大 B-Rep checkpoint 与增量保存？
- OCCT WASM 下 STEP/XCAF translation 对 10MB、100MB、500MB 模型的性能上限在哪里？
- 是否需要 worker / shared memory，把建模 kernel 从主线程完全隔离？
- persistent naming 采用 kernel 能力、应用层语义引用，还是二者结合？
- 原生 document 的版本迁移策略如何设计，才能不绑定具体 OCCT build？
- assembly instance 与几何共享怎样表达，才能避免复制 B-Rep？

下一步最值得做的实验

做一个真正的端到端 benchmark：浏览器导入中等复杂 STEP -> 转成 OCCT B-Rep -> tessellate -> 保存 native checkpoint -> 关闭并恢复 -> 做一次 Boolean/Fillet -> 再导出 STEP。记录时间、WASM 内存、文件大小、拓扑/几何差异。这会比继续讨论格式名称更快暴露架构瓶颈。

结论

Translation 的确绕不开，但它应该被限制在系统边界。Web CAD 内部应拥有自己的 canonical B-Rep 与 document semantics；用户需要互操作时，再将当前状态翻译成 STEP，或将 STEP 翻译成 canonical model。这样才能同时获得高效持续编辑、可演进的原生文档，以及开放 CAD 生态的互操作能力。